



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY Ramapuram

21CSS303T – DATA SCIENCE

Unit: I - Introduction to Data Science, Numpy and Pandas

Session title: "Foundations of Data Science with NumPy: A Beginner's Guide"

Session Outcome:x

By the end of this session, learners will be able to:

- i. Understand the basics of Data Science and its process
- ii. Learn more about facets of data science and their significance.
- iii. Explore **NumPy** for numerical computing and array manipulation.
- iv. Perform operations like **array creation, joining, splitting, searching, and sorting**.
- v. Apply **indexing, slicing, and shape manipulation** techniques on arrays.
- vi. Work with special arrays like the **identity array** and the **eye function**.

Data Science



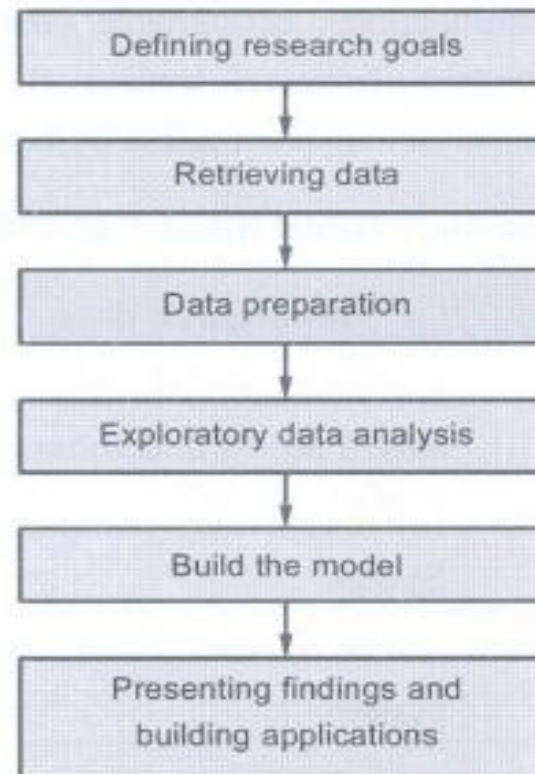
- **Data Science** is the field that involves using various techniques, algorithms, and systems to extract knowledge and insights from data.
- It blends skills from statistics, programming, and domain knowledge to analyze and interpret complex data to help make decisions or predictions.
- **For beginners, here is a simplified way to look at it:**
 1. **Data:** In the context of Data Science, data refers to raw facts and figures, which can be numbers, text, images, or other types of information.
 2. **Science:** Science refers to applying mathematical, statistical, and computational techniques to analyze and interpret the data.

Facets of data



- **Structured:** Data organized in rows and columns, e.g., a customer database with names and addresses.
- **Natural Language:** Human-readable text data that requires NLP techniques for processing.
- **Graph-Based:** Data represented as nodes and edges, e.g., a social network graph showing friendships.
- **Streaming:** Real-time data flow, e.g., stock market price updates or IoT sensor data.
- **Unstructured:** Data without a predefined format, e.g., videos, images, or raw text files.
- **Machine-Generated:** Data automatically created by machines, e.g., server logs or weather sensor readings.
- **Audio, Video, and Images:** Multimedia data, e.g., recorded podcasts, YouTube videos, or medical X-ray images.

Data Science Process



Data Science Process



Data Science Process Steps:

- **Discovery or Setting Research Goal:** Define the problem or objective clearly to guide the analysis.
- **Retrieve Data:** Collect relevant data from various sources such as databases, APIs, or web scraping.
- **Data Preparation:** Clean and preprocess the data to ensure quality and consistency..
- **Data Exploration:** Analyze the data to uncover patterns, trends, and relationships.
- **Data Modeling:** Develop predictive or descriptive models to solve the problem.
- **Presentation and Automation:** Communicate insights effectively and deploy automated solutions for real-world use.

Discovery or Defining research goal



Discovery or Defining research goal

- This step involves **defining the problem** or **objective** of the analysis.
- Understand the **business or research questions** to answer and define clear goals.
- **Example:**
 - For a retail business, the goal might be to predict customer churn or identify product recommendations.

Retrieving Data



Retrieving Data

- Gather data from **various sources**, including databases, APIs, open data platforms, web scraping, or manual collection.
- Ensure that the data collected is relevant to the defined problem.
- **Example:**
 - Downloading customer transaction data from a company's database or using APIs to gather real-time weather data.

Data Preparation



Data Preparation

- Clean and preprocess data to make it suitable for analysis..
- Ensure that the data collected is relevant to the defined problem.
- This step includes:
 - **Handling missing values**
 - **Removing duplicates**
 - **Standardizing formats**
 - **Encoding categorical variables**
- **Example:**
 - Replace null values in a dataset with the mean or median of the column.

Data Exploration



Data Exploration

- Perform **Exploratory Data Analysis (EDA)** to understand the structure, patterns, and relationships in the data.
- Use visualizations (e.g., histograms, scatter plots) and summary statistics.
- This step helps identify
 - **Outliers**
 - **Trends**
 - **Correlations**
- **Example:**
 - Analyzing sales trends over months to identify seasonality in the data.

Data Modeling



Data Modeling:

- Build machine learning or statistical models based on the processed data.
- Split the data into training, validation, and testing sets.
- Evaluate model performance using metrics like accuracy, precision, recall, or RMSE (Root Mean Square Error).
- **Example:**
 - Using logistic regression to predict whether a customer will buy a product (Yes/No).

Presentation and Automation



Presentation and Automation:

- Present insights and recommendations to stakeholders in a clear, understandable manner using dashboards, reports, or presentations.
- Automate the data pipelines or deploy the model for continuous use.
- **Example:**
 - Create an interactive Power BI dashboard to showcase sales predictions or deploy the model as an API for real-time predictions.

NumPy - Basics



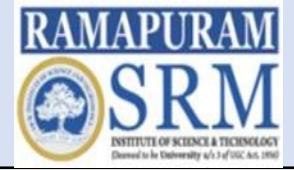
- NumPy - **N**umerical **P**ython - open source Python library widely used in science and engineering.
- NumPy library contains
 - multidimensional array data structures, such N-dimensional **ndarray**
 - a large set of library functions that operate efficiently on these data structures.

Installing and Importing NumPy



- NumPy can be installed using the command
pip install numpy
- After installing NumPy, it can be imported into Python code as
import numpy as <prefix>
import numpy as np
- NumPy features can now be accessed with the prefix (Eg: np).

NumPy vs Lists



NumPy	List
Contains only similar datatypes (Homogenous)	Can contains different types of datatypes (Heterogenous)
Element wise operation is possible.	Element wise operation is not possible on the list.
Using numpy.array(), N-Dimensional array can be created.	Stores elements as 1D list. Though N- Dimensional list can be created, it is a long process.
Occupies lesser memory and runs faster than lists.	Occupies more memory and runs slower than NumPy.
Each item is stored in a sequential manner.	It stores item in random location of the memory.

Creating NumPy Arrays from Lists

- Arrays can be created from lists. Example:

```
a = np.array([1, 2, 3, 4, 5, 6])  
print(a)
```

```
[1 2 3 4 5 6]
```

- Unlike Python lists, NumPy is constrained to contain the same type. If types do not match, NumPy will upcast if possible. Example:

```
a = np.array([3.14, 4, 2, 3])  
print(a)
```

```
[3.14 4.    2.    3.   ]
```

Creating NumPy Arrays from Lists

- Arrays can be created from lists. Example:

```
a = np.array([1, 2, 3, 4, 5, 6])  
print(a)  
  
[1 2 3 4 5 6]
```

- Unlike Python lists, NumPy is constrained to contain the same type. If types do not match, NumPy will upcast if possible. Example:

```
a = np.array([3.14, 4, 2, 3])  
print(a)  
  
[3.14 4.  2.  3. ]
```


Creating NumPy Arrays from Lists

- Arrays can be created from lists. Example:

```
a = np.array([1, 2, 3, 4, 5, 6])  
print(a)  
  
[1 2 3 4 5 6]
```

- Unlike Python lists, NumPy is constrained to contain the same type. If types do not match, NumPy will upcast if possible. Example:

```
a = np.array([3.14, 4, 2, 3])  
print(a)  
  
[3.14 4.    2.    3.   ]
```

..contd..

- The data type of the resulting array can be set explicitly using the keyword *dtype*.

Example:

```
a = np.array([1, 2, 3, 4], dtype=np.float32)
print(a)

[1.  2.  3.  4.]
```

- Multidimensional array can be created as list of lists. Example:

```
a = np.array([[2, 3, 4],[4, 5, 6],[6, 7, 8]])
print (a)

[[2 3 4]
 [4 5 6]
 [6 7 8]]
```

Creating NumPy Arrays from Lists

- Arrays can be created from lists. Example:

```
a = np.array([1, 2, 3, 4, 5, 6])  
print(a)  
  
[1 2 3 4 5 6]
```

- Unlike Python lists, NumPy is constrained to contain the same type. If types do not match, NumPy will upcast if possible. Example:

```
a = np.array([3.14, 4, 2, 3])  
print(a)  
  
[3.14 4.    2.    3.   ]
```

Creating NumPy Arrays using Functions

- One dimensional arrays can be created using *numpy.arange* function.
- *numpy.arange* creates arrays with regularly incrementing values.
- This function takes three inputs – start, end and step value (optional).

```
a = np.arange(10)
b = np.arange(10, 20)
c = np.arange(1, 20, 2)
print(a)
print(b)
print(c)
```



```
[0 1 2 3 4 5 6 7 8 9]
[10 11 12 13 14 15 16 17 18 19]
[ 1  3  5  7  9 11 13 15 17 19]
```

..contd..

- Multidimensional arrays can be created using *numpy.zeros* function, that will create an array filled with 0 values with the specified shape. The default dtype is float64. Example:

```
a = np.zeros((2, 3))  
b = np.zeros((2, 3, 2))  
print(a)  
print(b)
```

```
[[0. 0. 0.]  
 [0. 0. 0.]]  
[[[0. 0.]  
   [0. 0.]  
   [0. 0.]
```

```
[[0. 0.]  
 [0. 0.]  
 [0. 0.]
```

Guess the
output...

```
a = np.zeros((2, 3))  
print(a)  
b = np.zeros_like(a)  
print(b)
```

..contd..

- Similarly, *numpy.ones* will create an array filled with 1 values. It is identical to zeros in all other respects. Example:

```
a = np.ones((2, 3))  
b = np.ones((2, 3, 2))  
print (a)  
print (b)
```

```
[[1. 1. 1.]  
 [1. 1. 1.]]  
[[[1. 1.]  
   [1. 1.]  
   [1. 1.]
```

```
[[1. 1.]  
 [1. 1.]  
 [1. 1.]
```

Guess the
output...

```
a = np.ones((2, 3))  
print(a)  
b = np.ones_like(a)  
print(b)
```


- *numpy.full* will create an array filled with the specified value. Example:

```
a = np.full((3, 3), 3.14)
print(a)
```

```
[[3.14 3.14 3.14]
 [3.14 3.14 3.14]
 [3.14 3.14 3.14]]
```

- This code creates a 3*3 array, that is filled with the specified value – 3.14.

..contd..

- ***numpy.random.random*** creates an array with uniformly distributed pseudo-random values between 0 and 1.

```
a = np.random.random((3, 3))  
print(a)
```

```
[[0.33664967 0.89406689 0.70193815]  
 [0.95524235 0.55235533 0.86206224]  
 [0.03727202 0.84361484 0.29931326]]
```

..contd..

- *numpy.random.randint* creates an array of pseudo-random integers in the specified interval.

```
a = np.random.randint(0, 10, (3, 3))  
print(a)
```

```
[[5 1 5]  
 [7 9 2]  
 [8 6 6]]
```

- The above code created a 3*3 array with values between 0 and 10.

Explore...

- There are many other built-in routines to create array. Few of them are discussed here. Few other routines include:
 - ☐ `numpy.linspace`
 - ☐ `numpy.eye`
 - ☐ `numpy.diag`
 - ☐ `numpy.vander`
 - ☐ `numpy.vstack`
 - ☐ `numpy.hstack`
 - ☐ `numpy.block`
 - ☐ `numpy.copy`
 - ☐ `numpy.indices`

Restrictions of NumPy Arrays



- All elements of the array must be of the same type of data.
- Once created, the total size of the array can't be changed.
- The shape must be “rectangular”, not “jagged”; e.g., each row of a two-dimensional array must have the same number of columns.

NumPy Array Attributes

- Array attributes are properties of NumPy arrays that provide information about the array's shape, size, data type, dimension, and so on.

Attributes	Description
<code>ndim</code>	returns number of dimension of the array
<code>size</code>	returns number of elements in the array
<code>dtype</code>	returns data type of elements in the array
<code>shape</code>	returns the size of the array in each dimension.
<code>itemsize</code>	returns the size (in bytes) of each elements in the array
<code>data</code>	returns the buffer containing actual elements of the array in memory

..contd..

- The ***ndim*** attribute returns the number of dimensions in the numpy array.
- The ***size*** attribute returns the total number of elements in the given array.
- The ***shape*** attribute returns a tuple of integers that gives the size of the array in each dimension.
- The ***dtype*** attribute returns the datatype of a NumPy array.
- The ***itemsize*** attribute determines size (in bytes) of each element in the array.
- The ***data*** attribute returns the memory location where the array's data is stored in the computer's memory.

Example

```
a = np.array ([[2, 4, 6], [1, 3, 5], [9, 6, 9]])  
print("Dimension of the array is: ",a.ndim)  
print("Size of the array is: ",a.size)  
print("Shape of the array is: ",a.shape)  
print("Data type of the array is: ",a.dtype)  
print("Size of each element of the array is: ",a.itemsize)  
print("Memory location of the array is: ",a.data)
```

```
Dimension of the array is: 2  
Size of the array is: 9  
Shape of the array is: (3, 3)  
Data type of the array is: int64  
Size of each element of the array is: 8  
Memory location of the array is: <memory at 0x7b0d48436330>
```


NumPy Array Operations

- **Array Join:**Join multiple arrays along an axis.

```
arr1 = np.array([1, 2, 3])  
arr2 = np.array([4, 5, 6])  
result = np.concatenate((arr1, arr2))  
print(result)
```

```
[1 2 3 4 5 6]
```

- **Array Split:**Split a single array into multiple sub-arrays.

```
arr = np.array([1, 2, 3, 4, 5, 6])  
split_arr = np.array_split(arr, 3)  
print(split_arr)
```

```
[array([1, 2]), array([3, 4]), array([5, 6])]
```

NumPy Array Operations

- **Array Search:** Find the index positions where the condition is met.

```
arr = np.array([10, 20, 30, 40])
result = np.where(arr > 30)
print(result)

(array([3], dtype=int64),)
```

- **Array Split:** Sort the elements of an array in ascending order.

```
arr = np.array([4, 2, 3, 1])
sorted_arr = np.sort(arr)
print(sorted_arr)

[1 2 3 4]
```

Array Indexing

- Indexing is essential to access the array element based on the position. Array indexing starts from zero.
- Negative indices are used for accessing from the end of the array.

```
a = np.array ([1, 4, 9, 10, 12, 15])  
print ("First element of the array is: ",a[0])  
print ("Third element of the array is: ",a[2])  
print ("Last element of the array is: ",a[-1])  
print ("Second last element of the array is: ",a[-2])
```

```
First element of the array is: 1  
Third element of the array is: 9  
Last element of the array is: 15  
Second last element of the array is: 12
```

..contd..

- In a multi-dimensional array, items can be accessed using a comma-separated (row, column) tuple.

```
a = np.array([[1,2,3],[4,5,6],[7,8,9]])  
print(a[0,0]) # Access the element at row 0, column 0  
print(a[1,2]) # Access the element at row 1, column 2  
print(a[0,:]) # Access the first row  
print(a[:,0]) # Access the first column  
print(a[0:2, 1:3]) # Access a subarray
```

```
1  
6  
[1 2 3]  
[1 4 7]  
[[2 3]  
 [5 6]]
```

..contd..

- Array elements can be modified using the array index.

```
a = np.array([1, 2, 3, 4, 5])  
a[0] = 10 # Modify the first element  
a[2:4] = [20, 30] # Modify elements at index 2 and 3  
print(a)
```

```
[10  2 20 30  5]
```

Slicing



- ❖ Slicing is a Special Extraction function that allows user to extract desired part of any collection be it a List, Dictionary, String or Numpy array.
- ❖ It allows user to accessing a specified portion or subset of a collection for some action while leaving the rest of the list alone.

Slicing Syntax:

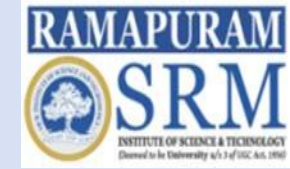


collection_name [start:stop:step]

- collection_name refers to any python containers
- start, stop are referred to the index in the specified containers which can either be positive or negative
- step - is the number increment between each index for slicing
- all the three parameters are optional, if start index doesn't specified its starts with „0“, if negative index doesn't specified, the length of the collection is considered, and default step is +1 if positive and -1 its negative.
- Also the stop index is always greater than the start index.

Note: If the start is not specified, it will be set to 0. If the end is not specified, it will run the length of the array in the dimension.

Selection statements



Index Position

0	1	2	3	4
1	2	3	4	5

Slice Position

0	1	2	3	4	5
-5	-4	-3	-2	-1	

The first row of numbers gives the position of the indices 0...5 in the string,
the second row gives the corresponding negative indices.

- ❖ `a[start:end]` # items from (start) to (end - 1)
- ❖ `a[start:]` # items from start through the rest of the array
- ❖ `a[:end]` # all items until and excluding (end)
- ❖ `a[:]` # a copy of the whole array
- ❖ `a[start:end:step]` #extract array elements in steps of the specified interval.

Slicing

- Three ways to perform Slicing
 - ❖ By providing only the start or stop parameters
 - ❖ By passing the parameters start and stop
 - ❖ By passing the parameters start, stop, and steps

Example:

```
▶ # importing numpy module
import numpy as np
# creating list
list = [1, 2, 3, 4, 5, 6]
# creating numpy array
arr = np.array(list)
#While Slicing ensuring the Stop index is always greater than Start
#Slicing Single Element
print("Element at Fourth Slice position is", arr[3])
```

Slicing

```
▶ #Slicing using Start and Stop  
print("Slicing from 1 to 3",arr[0:3])
```

⇒ Slicing from 1 to 3 [1 2 3]

```
[12] #Slicing using Start,End and Step by skipping in between elemnts  
print("Slicing Elements at Even position",arr[0:5:2])
```

⇒ Slicing Elements at Even position [1 3 5]

```
▶ #Slicing using only Start position  
print("Slicing from 3 position",arr[3:])
```

⇒ Slicing from 3 element [4 5 6]

```
[14] #Slicing using only Stop index  
print("Slicing using Stop position till 4",arr[:4])
```

⇒ Slicing using End position till 4 [1 2 3 4]

Slicing

▶ `#Slicing using without index`
`print("Slicing without any position",arr[:])`

⇒ Slicing without any position [1 2 3 4 5 6]

[16] `#Slicing using negative index`
`print("Slicing using neagtive index",arr[-4:-1])`

⇒ Slicing using neagtive index [3 4 5]

▶ `#Reverse the array using Slicing`
`print("Reverse array using Slicing",arr[::-1])`

⇒ Reverse array using Slicing [6 5 4 3 2 1]

Slicing – 2d array

- ❖ Slicing a 2D is quite same as 1D array, but here we have to specify the start, stop and step for each dimension of the array

Example:

```
arr_2D= [[1, 2, 3],[4, 5, 6]]
```

for the above 2D array we have to specify the slicing parameter for both the row and column

```
arr_2D[:2,:2]      #[1, 2], [4, 5]
```

1	2	3
4	5	6
7	8	9



1	2
3	4

#Slice a 2D array

```
arr_2D[:2,:2]
```

Slicing – 2d array

Example:

1	2	3
4	5	6
7	8	9



1	2
3	4

#Slice a 2D array

ar_2D[:2,:2]

```
list_2D = [[1, 2, 3],[4, 5, 6],[7, 8, 9]]  
arr_2D=np.array(list_2D)  
print(arr_2D[:2,:2])
```

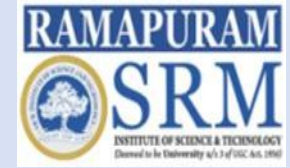
```
[[1 2]  
 [4 5]]
```

Slicing - Example

```
▶ arr_2D=np.array(list_2D)  
#Slicing the elements from 2 to 3 in Second row of a 2D array  
print(arr_2D[1,1:3])
```

```
➡ [5 6]
```

Iteration



- ❖ You can iterate over NumPy arrays just like Python lists, but NumPy allows for faster and more efficient iteration, especially with large datasets.
- ❖ **Example:** Iterating through a 1D array

```
In [19]: arr = np.array([1, 2, 3, 4, 5])  
         for x in arr:  
             print(x)
```

```
1  
2  
3  
4  
5
```

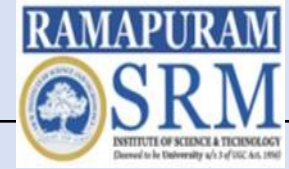
Iteration

❖ Example: Iterating through a 2D array

```
In [20]: arr_2d = np.array([[1, 2], [3, 4], [5, 6]])  
for row in arr_2d:  
    for element in row:  
        print(element)
```

```
1  
2  
3  
4  
5  
6
```


Practical application of Numpy



1. Data Preprocessing for Machine Learning

In machine learning, you often need to slice arrays to create training and test datasets or to normalize data.

Example: Splitting a dataset into training and testing sets

```
import numpy as np

# Assume arr is a dataset with 100 samples and 10 features
arr = np.random.rand(100, 10)

# Select the first 80 samples for training
train_data = arr[:80, :]

# Select the remaining 20 samples for testing
test_data = arr[80:, :]
```

Practical application of Numpy



Time Series Data Manipulation

When working with time series data, you may need to extract specific time windows or perform operations like rolling averages or filtering.

Example: Extracting a specific time range

```
# Time series data for a year (365 days, 1 variable)
arr = np.random.rand(365)
```

```
# Extract data from day 50 to day 150 (inclusive)
time_range = arr[50:151]
```

Practical application of Numpy

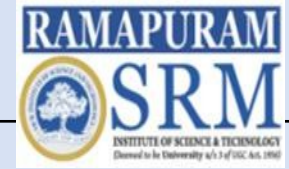


Image Processing

In image processing, NumPy arrays are used to represent images, where you can slice sections of the image (such as regions of interest or sub-images) for various transformations or filters.

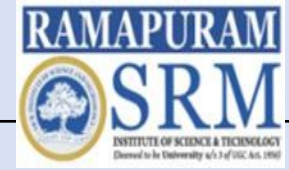
Example: Cropping an image

```
import numpy as np

# Create a 3x3 "image" (3x3 pixels, 3 color channels)
image = np.random.rand(10, 10, 3)

# Extract a 3x3 region from the image
cropped_image = image[2:5, 2:5, :] # Crops a 3x3 region from (2, 2)
to (5, 5)
```

Practical application of Numpy



Data Augmentation for Deep Learning

In deep learning tasks like image processing, you may slice data for augmentation, such as flipping or rotating images, or adding noise to training samples.

Example: Flipping images horizontally

```
# Create a random 5x5 image (grayscale)
image = np.random.rand(5, 5)

# Flip the image horizontally
flipped_image = image[:, ::-1]
```

Batch Processing

In deep learning, you often need to process data in batches. You can slice data into batches for efficient training.

Example: Splitting data into batches

```
# Assume we have 100 samples and we want to split them into 5
batches
arr = np.random.rand(100, 10)

batch_size = 20
batches = [arr[i:i+batch_size, :] for i in range(0, len(arr),
batch_size)]
```

Copying Arrays in NumPy



- In NumPy, array assignment does not create a new array, but rather a reference to the original array. To avoid this, you can create a copy of the array.
- **Array Copying:**
 - **Shallow copy:** This creates a new array, but the elements in the array still reference the same memory locations.
 - **Deep copy:** This creates a new array and copies the data of the original array entirely into the new array.

Example Of Shallow and Deep Array



- Shallow copy using copy():

```
arr = np.array([1, 2, 3])
arr_copy = arr
arr_copy[0] = 10
print(arr)      # Output: [10 2 3]
print(arr_copy) # Output: [10 2 3]
```

- Deep copy using copy():

```
arr = np.array([1, 2, 3])
arr_copy = arr.copy()
arr_copy[0] = 10
print(arr)      # Output: [1 2 3]
print(arr_copy) # Output: [10 2 3]
```

- The **deep copy** method ensures that c the original array.

Example Of Shallow and Deep Array

Creating copy of original data

- In Python, there are two ways to create copies
 - Shallow copy
 - Deep copy

	Shallow copy	Deep copy
Function	<pre>samp=cars_data.copy(deep=False)</pre> <pre>samp = cars_data</pre>	<pre>cars_data1=cars_data.copy(deep=True)</pre>
Description	◦ It only creates a new variable that shares the reference of the original object ◦ Any changes made to a copy of object will be reflected in the original object as well	◦ In case of deep copy, a copy of object is copied in other object with no reference to the original ◦ Any changes made to a copy of object will not be reflected in the original object

Array Shape Manipulation



- **Reshaping arrays** allows you to change the dimensions of an array without changing its data.

NumPy provides several functions for reshaping, such as reshape, resize and flatten

- **Reshape:** It allows you to change the shape of an array.

```
In [3]: arr = np.array([1, 2, 3, 4, 5, 6])  
        reshaped_arr = arr.reshape(2, 3)  
        print(reshaped_arr)
```

```
[[1 2 3]  
 [4 5 6]]
```

Array Shape Manipulation



- **Flatten:** It converts a multi-dimensional array into a 1D array.

```
In [4]: arr = np.array([[1, 2], [3, 4], [5, 6]])  
        flattened_arr = arr.flatten()  
        print(flattened_arr)
```

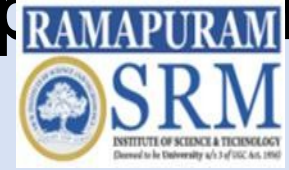
[1 2 3 4 5 6]

- **Resize:** It changes the shape of an array and can fill the new array with values.

```
In [5]: arr = np.array([1, 2, 3, 4])  
        arr.resize((2, 2))  
        print(arr)
```

[[1 2]
 [3 4]]

Practical application of Numpy Array shape Manipulation



Data Reshaping for Machine Learning

In machine learning, the data often needs to be reshaped to meet the input requirements of a model (e.g., for multi-dimensional input).

Example: Flattening images for Neural Networks

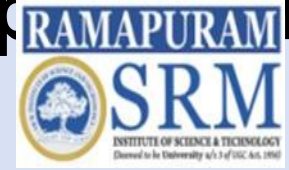
When using Convolutional Neural Networks (CNNs), images are typically represented as multi-dimensional arrays (e.g., height x width x channels), but fully connected layers require a flat vector as input.

```
import numpy as np

# Example image (28x28 pixels, 1 color channel)
image = np.random.rand(28, 28)

# Flatten the image to a 1D vector for a fully connected layer
flattened_image = image.reshape(-1) # shape becomes (784,)
```

Practical application of Numpy Array shape Manipulation



Handling Time Series Data

Time series data often comes in 1D or 2D arrays, but many machine learning models expect data in a 3D format (samples, timesteps, features). You can manipulate array shapes to convert your data into the required format.

Example: Preparing time-series data for LSTM

In LSTM networks, you need to provide the data in the form of (samples, timesteps, features).

```
# 1D array of 100 time steps, each with 1 feature
time_series = np.random.rand(100)

# Reshape into 2D: (100 samples, 1 timestep, 1 feature)
time_series_resaped = time_series.reshape(100, 1, 1)
```

Practical application of Numpy Array shape Manipulation

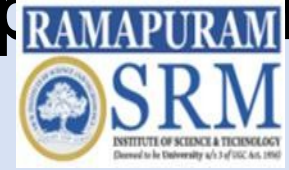


Image or Video Processing

In image and video processing, manipulating the shape of arrays allows you to extract frames from a video, crop regions of an image, or resize images.

Example: Reshaping frames for video processing

If you are processing a video, each frame may be represented as a 2D array, but to manipulate the video as a collection, you need to stack the frames into a 3D array.

```
# Example: Stacking frames of a video
frames = np.random.rand(10, 64, 64, 3) # 10 frames, 64x64 pixels, 3 color channels

# Access a single frame
frame_5 = frames[4] # shape: (64, 64, 3)
```

Identity Arrays



- An **identity array** is a square array where the diagonal elements are 1 and all other elements are 0

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Identity Arrays

- **numpy.identity(n, dtype = None)** : Return a identity matrix i.e. a square matrix with ones on the main diagonal
- Parameters:
 - **n** : [int] Dimension n x n of output array
 - **dtype** : [optional, float(by Default)] Data type of returned array
- Creating Identity Arrays: Use **np.eye()** to create an identity array

```
In [6]: identity_arr = np.eye(3)  
print(identity_arr)
```

```
[[1.  0.  0.]  
 [0.  1.  0.]  
 [0.  0.  1.]]
```

Eye Function



- Creating Identity Arrays: The **np.eye()** function in NumPy is used to create a 2D array (matrix) where all the diagonal elements are set to 1
- This is commonly known as an **identity matrix**, which is widely used in linear algebra and various matrix operations.
- **Syntax:**

python

Copy code

```
numpy.eye(N, M=None, k=0, dtype=float, order='C')
```


Eye Function



- **numpy.eye(R, C = None, k = 0, dtype = type <'float'>)** :
Return a matrix having 1's on the diagonal and 0's elsewhere w.r.t. k.
- **R** : Number of rows
C : [optional] Number of columns; By default M = N
k : [int, optional, 0 by default]
Diagonal we require; k>0 means diagonal above main diagonal or vice versa.
dtype : [optional, float(by Default)] Data type of returned array.

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Eye Function

✓
0s

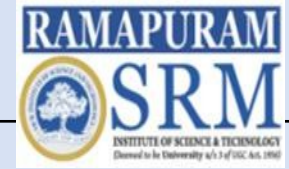


```
import numpy as np
print(np.eye(4))
print(np.eye(3,2))
print(np.eye(3,3,1))
print(np.eye(3,2,-1))
```



```
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
[[1.  0.]
 [0.  1.]
 [0.  0.]]
[[0.  1.  0.]
 [0.  0.  1.]
 [0.  0.  0.]]
[[0.  0.]
 [1.  0.]
 [0.  1.]]
```

Practical application of Numpy Identity Arrays



Regularization in Machine Learning (e.g., Ridge Regression):

In some machine learning models, such as Ridge Regression, an identity matrix is used to regularize the model and prevent overfitting. In this case, the identity matrix is scaled by a regularization parameter and added to the covariance matrix.

```
from numpy.linalg import inv
X = np.array([[1, 2],
              [3, 4],
              [5, 6]])
y = np.array([1, 2, 3])
# Regularization parameter
alpha = 1.0
# Identity matrix scaled by alpha
I = np.eye(X.shape[1])
# Ridge regression closed-form solution
XTX = np.dot(X.T, X)
XTy = np.dot(X.T, y)
ridge_solution = inv(XTX + alpha * I).dot(XTy)
print(ridge_solution)
```